



## รายงานฉบับสมบูรณ์

AI-Assisted Testing vs. Automatic Test Case  
Generation Algorithms:  
A Benchmark and Test Coverage Evaluation

จัดทำโดย

นางสาวกัญญาวี ศรีहरา 673380026-6

นางสาวรสริน เมืองหงษ์ 673380289-4

นางสาวสโรชา เสาทอง 673380296-7

อาจารย์ผู้สอน

ผศ. ดร.ชิตสุธา สุ่มเล็ก

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance

ภาคเรียนที่ 1 ประจำปีการศึกษา 2569

สาขาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

## คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance โดยมีจุดมุ่งหมายเพื่อศึกษาและเปรียบเทียบการสร้าง Test Case อัตโนมัติด้วยอัลกอริทึมค้นหา ได้แก่ Simulated Annealing (SA) และ Binary Particle Swarm Optimization (BPSO) กับการสร้าง Test ด้วย Generative AI ได้แก่ ChatGPT และ Gemini โดยทดลองกับบักจิ้งใน Defects4J dataset และวัดผลด้วย Fault Detection, Line Coverage, Condition/Branch Coverage และเวลาที่ใช้สร้าง Test

เนื้อหาแบ่งเป็น 5 บท เริ่มจากที่มาและวัตถุประสงค์ ทฤษฎีและงานที่เกี่ยวข้อง วิธีการดำเนินงาน

ผลการทดลองและการวิเคราะห์ และสรุปผลพร้อมข้อเสนอแนะ

คณะผู้จัดทำหวังว่ารายงานนี้จะเป็นประโยชน์ต่อผู้สนใจการสร้าง Test อัตโนมัติ

และขอขอบคุณอาจารย์ผู้สอนที่ให้คำแนะนำตลอดการทำโครงการ

คณะผู้จัดทำ

4 ตุลาคม 2569

## สารบัญ

|                                                                                  | หน้า      |
|----------------------------------------------------------------------------------|-----------|
| คำนำ                                                                             | 2         |
| สารบัญ                                                                           | 3         |
| สารบัญ(ต่อ)                                                                      | 4         |
| สารบัญรูปภาพ                                                                     | 5         |
| สารบัญตาราง                                                                      | 6         |
| <b>บทที่ 1 บทนำ</b>                                                              | <b>7</b>  |
| 1.1 ที่มาและความสำคัญ                                                            | 7         |
| 1.2 วัตถุประสงค์                                                                 | 7         |
| 1.3 ขอบเขตของโครงการ                                                             | 8         |
| <b>บทที่ 2 เอกสารและงานวิจัยที่เกี่ยวข้อง</b>                                    | <b>9</b>  |
| 2.1 การทดสอบซอฟต์แวร์และการสร้างกรณีทดสอบอัตโนมัติ                               | 9         |
| 2.2 อัลกอริทึมการค้นหาและการปรับแต่ง (Metaheuristic Optimization)                | 9         |
| 2.2.1 Simulated Annealing (SA)                                                   | 10        |
| 2.2.2 Binary Particle Swarm Optimization (BPSO)                                  | 11        |
| 2.3 การประยุกต์ใช้ Generative AI ในการทดสอบซอฟต์แวร์                             | 12        |
| 2.4 ชุดข้อมูล Defects4J และตัวชี้วัดประสิทธิภาพ                                  | 12        |
| <b>บทที่ 3 วิธีการดำเนินงานวิจัย</b>                                             | <b>13</b> |
| 3.1 ภาพรวมของสถาปัตยกรรมระบบและกระบวนการทำงาน                                    | 13        |
| 3.2 การจัดเตรียมชุดข้อมูลและการคัดเลือก Java Projects จาก Defects4J              | 22        |
| 3.3 การออกแบบและพัฒนา Automated Test Case Generation (SA และ BPSO)               | 23        |
| 3.4 การออกแบบ Prompt Engineering สำหรับ AI-Assisted Testing (ChatGPT และ Gemini) | 24        |
| 3.5 การออกแบบระบบ TestBench                                                      | 24        |

## สารบัญ(ต่อ)

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| <b>บทที่ 4 ผลการศึกษาและวิเคราะห์ผล</b>                                                | <b>26</b> |
| 4.1 ผลการทดสอบระดับความครอบคลุมของโค้ด (Code Coverage Evaluation)                      | 26        |
| 4.2 ผลการทดสอบความสามารถในการตรวจจับข้อผิดพลาด (Fault Detection Rate)                  | 26        |
| 4.3 การเปรียบเทียบระยะเวลาและทรัพยากรในการสร้าง Test Case                              | 27        |
| 4.4 วิเคราะห์เปรียบเทียบเชิงลึกระหว่าง AI-Assisted Testing กับ Optimization Algorithms | 27        |
| <b>บทที่ 5 สรุปผลการศึกษาและข้อเสนอแนะ</b>                                             | <b>28</b> |
| 5.1 สรุปผลการดำเนินงาน                                                                 | 28        |
| 5.2 ข้อดีและข้อจำกัดของแต่ละแนวทาง                                                     | 28        |
| 5.3 ปัญหาและอุปสรรคในการดำเนินงาน                                                      | 29        |
| 5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อในอนาคต                                              | 29        |
| <b>เอกสารอ้างอิง</b>                                                                   | <b>30</b> |

## สารบัญรูปภาพ

| รูปภาพ                                                      | หน้า |
|-------------------------------------------------------------|------|
| รูปภาพที่ 1 ภาพรวมระบบ                                      | 13   |
| รูปภาพที่ 2 ขั้นตอนของหนึ่ง run (ต่อหนึ่ง bug ต่อหนึ่งรอบ)  | 15   |
| รูปภาพที่ 3 Pipeline ของ algorithm (SA และ BPSO ใช้ร่วมกัน) | 16   |
| รูปภาพที่ 4 Simulated Annealing                             | 17   |
| รูปภาพที่ 5 Binary PSO                                      | 17   |
| รูปภาพที่ 6 Pipeline ของ AI (ต่อ Ai หนึ่งตัว)               | 18   |
| รูปภาพที่ 7 การตัดสินใจผลหลังรัน buggy และ fixed            | 19   |
| รูปภาพที่ 8 Campaign (ทั้ง dataset)                         | 20   |
| รูปภาพที่ 9 การประกอบ prompt ของ AI                         | 21   |

## สารบัญตาราง

| ตาราง                                                              | หน้า |
|--------------------------------------------------------------------|------|
| ตารางที่ 1 ตัวชี้วัดที่ใช้ในการประเมิน                             | 12   |
| ตารางที่ 2 ตารางภาพรวมระบบ                                         | 14   |
| ตารางที่ 3 การออกแบบ Prompt Engineering สำหรับ AI-Assisted Testing | 24   |
| ตารางที่ 4 ผลตัดสินจากการเทียบผล Buggy กับ Fixed                   | 25   |
| ตารางที่ 5 ค่ากำหนดการทดลอง                                        | 25   |
| ตารางที่ 6 ผลการทดลองของทั้ง 4 วิธี (ทดสอบบน 854 บั๊กเท่ากัน)      | 26   |
| ตารางที่ 7 ผลต่างของแต่ละวิธีเมื่อเทียบกับ SA                      | 26   |
| ตารางที่ 8 ข้อดีและข้อจำกัดของแต่ละแนวทาง                          | 28   |

## บทที่ 1

### บทนำ

#### 1.1 ที่มาและความสำคัญ

การทดสอบซอฟต์แวร์ (Software Testing)

เป็นกระบวนการสำคัญในการตรวจสอบคุณภาพและค้นหาข้อบกพร่องของโปรแกรม

โดยเฉพาะการทดสอบระดับหน่วย (Unit Testing) ซึ่งตรวจสอบการทำงานของ Class หรือ Method แต่ละส่วน อย่างไรก็ตาม การสร้าง Test Case

ด้วยตนเองใช้เวลามากและต้องเข้าใจโครงสร้างและพฤติกรรมของ Source Code

ผู้พัฒนาหรือผู้ทดสอบจึงอาจสร้าง Test Case ที่ครอบคลุมทุกเงื่อนไขและทุกเส้นทางการทำงานไม่ได้

แนวทางหนึ่งที่จะช่วยลดภาระนี้คือ Automated Test Case Generation ซึ่งใช้ Algorithm ค้นหาและสร้าง Test Input และ Test Case โดยอัตโนมัติ โครงงานนี้เลือกศึกษา Simulated Annealing (SA) และ Binary Particle Swarm Optimization (BPSO) ซึ่งเป็น Metaheuristic Optimization Algorithm ที่นำมาใช้กับปัญหาการสร้างข้อมูลทดสอบได้ โดยกำหนด Test Input หรือชุด Test เป็น Candidate Solution และใช้ Fitness Function ประเมินคุณภาพตามเป้าหมายของการทดสอบ

ขณะเดียวกัน Generative AI และ AI-Assisting Tools

เข้ามามีบทบาทในการพัฒนาซอฟต์แวร์มากขึ้น AI สามารถวิเคราะห์ Source Code และสร้าง Test Case ตาม Prompt ที่กำหนดได้ แต่คุณภาพของ Test ที่ได้ขึ้นกับข้อมูลที่ให้ รูปแบบของ Prompt และความสามารถของเครื่องมือ Test ที่ AI สร้างจึงต้องถูกนำไปรันจริงกับโปรแกรมเป้าหมายเพื่อตรวจสอบ

โครงงานนี้จึงศึกษาและเปรียบเทียบประสิทธิภาพของการสร้าง Test Case ด้วย SA, BPSO, ChatGPT และ Gemini ในการสร้าง Test Suite ระดับ Unit Testing สำหรับ Java Projects ใน Defects4J จากนั้นนำ Test ที่ได้ไปตรวจสอบและประเมินด้วย Test Coverage, Code Coverage Ratio, Fault Detection Rate และเวลาในการสร้าง เพื่อดูข้อดี ข้อจำกัด และความเหมาะสมของแต่ละแนวทาง

#### 1.2 วัตถุประสงค์

1.2.1 เพื่อศึกษาหลักการและกระบวนการทำงานของ Automated Test Case Generation Algorithms ได้แก่ SA และ BPSO

1.2.2 เพื่อศึกษาการใช้ Generative AI ได้แก่ ChatGPT และ Gemini สำหรับสร้าง Test Suite และ Test Case ระดับ Unit Testing

1.2.3 เพื่อออกแบบและพัฒนาแนวทางนำ SA และ BPSO มาสร้าง Test Input และ Test Case สำหรับ Java Projects ใน Defects4J

1.2.4 เพื่อออกแบบ Prompt สำหรับสั่ง ChatGPT และ Gemini ให้สร้าง Unit Test ที่เหมาะกับ Java Source Code และ Method ที่กำหนด

1.2.5 เพื่อเปรียบเทียบความถูกต้องและความสามารถในการครอบคลุมโค้ดของ Test ที่สร้างจากอัลกอริทึมและจาก Generative AI

1.2.6 เพื่อประเมินประสิทธิภาพของแต่ละแนวทางด้วยตัวชี้วัด Test Coverage, Code Coverage Ratio, Fault Detection Rate จำนวน Test ที่สร้างได้ และเวลาในการสร้าง

1.2.7 เพื่อวิเคราะห์ข้อดี ข้อจำกัด และความเหมาะสมของทั้ง 4 วิธีสำหรับการสร้าง Test Case อัตโนมัติ

### 1.3 ขอบเขตของโครงการ

ด้านอัลกอริทึม: ศึกษาและใช้งาน Simulated Annealing (SA) และ Binary Particle Swarm Optimization (BPSO)

ด้านเครื่องมือ AI: ใช้ ChatGPT และ Gemini เจนเนอเรต Unit Test Case

ด้านชุดข้อมูล: ใช้ Java Projects จาก Defects4J dataset

ด้านการวัดผล: ประเมินด้วย Code Coverage, Fault Detection Rate และเวลาที่ใช้

## บทที่ 2

### เอกสารและงานวิจัยที่เกี่ยวข้อง

#### 2.1 การทดสอบซอฟต์แวร์และการสร้างกรณีทดสอบอัตโนมัติ

การทดสอบหน่วย (Unit Testing) เป็นกระบวนการทดสอบส่วนย่อยของโปรแกรม เช่น เมธอด (Method) หรือคลาส (Class) โดยแยกส่วนที่ต้องการทดสอบออกจากส่วนอื่นของระบบ เพื่อประเมินว่าส่วนดังกล่าวสามารถทำงานได้ถูกต้องตามข้อกำหนดที่กำหนดไว้หรือไม่ ในภาษา Java นิยมใช้เฟรมเวิร์ก JUnit สำหรับการเขียนและดำเนินการทดสอบ โดยกรณีทดสอบ (Test Case) ที่ดีควรกำหนดข้อมูลนำเข้า (Input) ดำเนินการเรียกส่วนของโปรแกรมที่ต้องการทดสอบ และตรวจสอบผลลัพธ์ที่ได้เปรียบเทียบกับผลลัพธ์ที่คาดหวัง (Expected Output)

การสร้างกรณีทดสอบอัตโนมัติ (Automated Test Case Generation) เป็นกระบวนการที่ใช้ซอฟต์แวร์หรืออัลกอริทึมในการสร้างกรณีทดสอบโดยอัตโนมัติ แทนการออกแบบและเขียนกรณีทดสอบด้วยตนเอง วิธีการดังกล่าวช่วยลดเวลาและแรงงานในการสร้างกรณีทดสอบ และสามารถสร้างกรณีทดสอบจำนวนมากได้อย่างรวดเร็ว อย่างไรก็ตาม ความท้าทายสำคัญคือการประเมินคุณภาพของกรณีทดสอบที่สร้างขึ้น เนื่องจากจำนวนกรณีทดสอบเพียงอย่างเดียวไม่สามารถสะท้อนประสิทธิภาพของชุดทดสอบได้ จึงจำเป็นต้องกำหนดเกณฑ์สำหรับประเมินคุณภาพอย่างชัดเจน เช่น ระดับความครอบคลุมของโค้ด (Code Coverage) และความสามารถในการตรวจจับข้อผิดพลาดหรือบั๊ก (Defect Detection)

ในโครงงานนี้ แนวทางการสร้างกรณีทดสอบอัตโนมัติแบ่งออกเป็น 2 แนวทางหลัก ได้แก่ แนวทางการค้นหาแบบอิงการค้นหา (Search-Based Approach) ซึ่งใช้อัลกอริทึมในการค้นหาและปรับปรุงคำตอบอย่างต่อเนื่องเพื่อให้ได้ชุดกรณีทดสอบที่มีคุณภาพ และแนวทางที่ใช้ Generative AI ซึ่งอาศัยแบบจำลองปัญญาประดิษฐ์เชิงสร้างสรรค์ในการวิเคราะห์ข้อมูลที่กำหนดผ่าน Prompt และสร้างกรณีทดสอบตามข้อมูลและเงื่อนไขที่ระบุ

## 2.2 อัลกอริทึมการค้นหาและการปรับแต่ง (Metaheuristic Optimization)

ปัญหาการสร้างกรณีทดสอบสามารถพิจารณาเป็นปัญหาการหาค่าที่เหมาะสม (Optimization Problem) เนื่องจากจำนวนชุดกรณีทดสอบที่สามารถสร้างขึ้นได้มีจำนวนมาก และไม่สามารถทดลองชุดคำตอบที่เป็นไปได้ทั้งหมดภายในเวลาที่จำกัดได้ ดังนั้น จึงสามารถนำวิธีการแบบเมตาฮิวริสติก (Metaheuristic) มาใช้ในการค้นหาคำตอบที่มีคุณภาพภายในพื้นที่คำตอบขนาดใหญ่ โดยไม่จำเป็นต้องตรวจสอบคำตอบที่เป็นไปได้ทั้งหมด

ในการประยุกต์ใช้กับการสร้างกรณีทดสอบ แต่ละคำตอบที่เป็นไปได้ (Candidate Solution) สามารถแทนด้วยชุดกรณีทดสอบหนึ่งชุด และใช้ฟังก์ชันความเหมาะสม (Fitness Function) เป็นเกณฑ์ในการประเมินคุณภาพของชุดกรณีทดสอบนั้น โดยอัลกอริทึมจะค้นหาและปรับปรุง Candidate Solution อย่างต่อเนื่อง เพื่อให้ได้คำตอบที่มีค่า Fitness ตามเป้าหมายที่กำหนด

### 2.2.1 Simulated Annealing (SA)

เป็นอัลกอริทึมการค้นหาแบบเมตาฮิวริสติกที่ได้รับแรงบันดาลใจจากกระบวนการหลอมโลหะ โดยเริ่มต้นจากคำตอบหนึ่งคำตอบและสร้างคำตอบข้างเคียง (Neighbor Solution) ด้วยการปรับเปลี่ยนคำตอบเดิมในระดับเล็กน้อย จากนั้นจึงเปรียบเทียบค่าความเหมาะสมของคำตอบใหม่กับคำตอบปัจจุบัน หากคำตอบใหม่มีค่าความเหมาะสมดีกว่าคำตอบปัจจุบัน อัลกอริทึมจะยอมรับคำตอบใหม่ทันที ในทางกลับกัน หากคำตอบใหม่มีค่าความเหมาะสมแย่ลง อัลกอริทึมยังสามารถยอมรับคำตอบดังกล่าวได้ด้วยความน่าจะเป็น ซึ่งช่วยให้อัลกอริทึมสามารถหลีกเลี่ยงการติดอยู่ในจุดเหมาะสมเฉพาะที่ (Local Optimum) และเพิ่มโอกาสในการค้นหาคำตอบที่ดีกว่าในพื้นที่อื่นของปริภูมิคำตอบ ความน่าจะเป็นในการยอมรับคำตอบที่มีค่าความเหมาะสมแย่ลงสามารถคำนวณได้จากสมการ

$$P = e^{-(\Delta/T)}$$

โดยที่ ค่า  $\Delta$  คือผลต่างของค่าความเหมาะสมระหว่างคำตอบใหม่และคำตอบปัจจุบัน ซึ่งจะมีค่าเป็นลบเมื่อคำตอบใหม่มีคุณภาพแย่ลง และ (T) คือค่าอุณหภูมิ (Temperature)

ที่ใช้ควบคุมความน่าจะเป็นในการยอมรับคำตอบที่แย่ง ในช่วงเริ่มต้นของกระบวนการ ค่า (T) จะอยู่ในระดับสูง ส่งผลให้มีโอกาสยอมรับคำตอบที่มีค่าความเหมาะสมแย่งได้มาก ทำให้อัลกอริทึมสามารถสำรวจพื้นที่คำตอบได้อย่างกว้างขวาง เมื่อกระบวนการค้นหาดำเนินไป ค่า (T) จะค่อย ๆ ลดลง ส่งผลให้ความน่าจะเป็นในการยอมรับคำตอบที่แย่งลดลงตามไปด้วย ทำให้การค้นหาในช่วงท้ายมุ่งเน้นไปที่การปรับปรุงคำตอบที่มีคุณภาพอยู่แล้วมากขึ้น กลไกดังกล่าวช่วยเพิ่มความสามารถในการหลีกเลี่ยง Local Optimum และเพิ่มโอกาสในการค้นหาคำตอบที่เหมาะสมกว่า

## 2.2.2 Binary Particle Swarm Optimization (BPSO)

Binary Particle Swarm Optimization (BPSO) เป็นอัลกอริทึมที่พัฒนาต่อยอดจาก Particle Swarm Optimization (PSO) โดย PSO เป็นอัลกอริทึมการค้นหาแบบกลุ่ม (Population-Based Optimization) ที่จำลองแนวคิดการเคลื่อนที่และการเรียนรู้ร่วมกันของกลุ่มสิ่งมีชีวิต เช่น ผีเสื้อหรือฝูงปลา โดยแต่ละ Particle จะปรับปรุงตำแหน่งของตนเองจากประสบการณ์ที่ผ่านมาและข้อมูลจาก Particle อื่นภายในกลุ่ม

โดยทั่วไป PSO ใช้ตำแหน่งของ Particle ในรูปแบบค่าต่อเนื่อง (Continuous Values) ขณะที่ BPSO ปรับรูปแบบการแทนตำแหน่งให้เป็นค่าฐานสอง (Binary Values) ซึ่งประกอบด้วย 0 และ 1 จึงเหมาะกับปัญหาที่มีลักษณะการตัดสินใจแบบเลือกหรือไม่เลือก เช่น การเลือกกว่ากรณีทดสอบใดควรถูกรวมไว้ในชุดกรณีทดสอบ ดังนั้น BPSO จึงเป็นรูปแบบที่ดัดแปลงจาก PSO เพื่อรองรับปัญหาการค้นหาในปริภูมิแบบไบนารี แต่ละ Particle จะเก็บข้อมูลสำคัญ 2 ค่า ได้แก่ **pBest (Personal Best)** ซึ่งเป็นคำตอบที่ดีที่สุดที่ Particle นั้นเคยพบจากประสบการณ์ของตนเอง และ **gBest (Global Best)** ซึ่งเป็นคำตอบที่ดีที่สุดที่พบโดย Particle ทั้งหมดภายในกลุ่ม การปรับค่าความเร็ว (Velocity) ของ Particle สามารถอธิบายได้ด้วยสมการ

$$v = wv + c_1r_1(pBest-x) + c_2r_2(gBest-x)$$

โดยที่ (v) คือค่าความเร็วของ Particle, (w) คือค่าน้ำหนักความเฉื่อย (Inertia Weight), ( $c_1$ ) และ ( $c_2$ ) คือค่าสัมประสิทธิ์การเรียนรู้จากประสบการณ์ส่วนบุคคลและประสบการณ์ของกลุ่มตามลำดับ, ( $r_1$ ) และ ( $r_2$ ) คือค่าตัวเลขสุ่มในช่วง 0 ถึง 1, (pBest) คือคำตอบที่ดีที่สุดของ Particle นั้น, (gBest) คือคำตอบที่ดีที่สุดของกลุ่ม และ (x) คือตำแหน่งปัจจุบันของ Particle เนื่องจาก BPSO

ใช้ตำแหน่งในรูปแบบไบนารี จึงต้องแปลงค่าความเร็วให้เป็นความน่าจะเป็นของการกำหนดค่าบิต โดยใช้ฟังก์ชัน Sigmoid ดังสมการ

$$S(v) = 1 / (1 + e^{(-v)})$$

ค่าที่ได้จากฟังก์ชัน Sigmoid จะอยู่ในช่วง 0 ถึง 1

และสามารถตีความเป็นความน่าจะเป็นที่ตำแหน่งนั้นจะมีค่าเป็น 1

จากนั้นจึงใช้ค่าความน่าจะเป็นดังกล่าวในการสุ่มกำหนดค่าของบิตเป็น 0 หรือ 1

แนวคิดสำคัญของ BPSO คือการค้นหาคำตอบโดยอาศัยทั้งประสบการณ์ของแต่ละ Particle

และข้อมูลจากกลุ่ม โดย Particle จะพยายามเคลื่อนที่ไปยังบริเวณที่เคยให้คำตอบที่ดีสำหรับตนเอง

ขณะเดียวกันก็เรียนรู้จากคำตอบที่ดีที่สุดของกลุ่ม

ทำให้สามารถสำรวจพื้นที่คำตอบและปรับปรุงคุณภาพของคำตอบอย่างต่อเนื่อง

## 2.3 การประยุกต์ใช้ Generative AI ในการทดสอบซอฟต์แวร์

Generative AI อย่าง ChatGPT และ Gemini เป็นโมเดลภาษาขนาดใหญ่ที่รับข้อความเป็นคำสั่ง (Prompt) แล้วสร้างข้อความหรือโค้ดตอบกลับ ในงานทดสอบ เมื่อให้ Source Code ข้อมูลของบัก และข้อกำหนดการทดสอบ โมเดลจะสร้างโค้ด JUnit Test ได้ จุดเด่นคือเข้าใจความหมายของโค้ดและสร้าง Test ที่อ่านง่ายได้ จุดอ่อนคือ Test ที่สร้างอาจเรียก Method หรือ Constructor ไม่ตรงกับ Source จริง หรือ Import ผิด จึงต้องนำไป Compile และรันเพื่อตรวจสอบเสมอ งานนี้ใช้ ChatGPT และ Gemini ผ่าน Openrouter, Local AI

## 2.4 ชุดข้อมูล Defects4J และตัวชี้วัดประสิทธิภาพ

Defects4J คือ คลังบั๊กจริงของโปรเจกต์ Java โอเพนซอร์ส ประกอบด้วย 17 Project ได้แก่ Chart, Cli, Closure, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang, Math, Mockito และ Time แต่ละบั๊กมี 2 เวอร์ชัน คือ Buggy (โค้ดที่มีบั๊ก) และ Fixed (โค้ดหลังแก้) การตรวจว่า Test จับ Bug ได้หรือไม่ ทำโดยรัน Test บนทั้งสองเวอร์ชันแล้วเทียบผล

| ตัวชี้วัด                   | ความหมาย                                | วิธีคำนวณ                                                                                                                        |
|-----------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| FDR                         | ความสามารถของ Test ในการตรวจจับ Bug     | $\text{Detection Rate} = \frac{\text{จำนวน Test ที่ Buggy Fail และ Fixed Pass}}{\text{จำนวน Test ที่ประเมินทั้งหมด}} \times 100$ |
| Line Coverage               | สัดส่วนบรรทัดโค้ดที่ Execute            | $(\text{จำนวนบรรทัดที่ถูก Execute} / \text{จำนวนบรรทัดทั้งหมด}) \times 100$                                                      |
| Condition / Branch Coverage | สัดส่วนเงื่อนไขและเส้นทางที่ถูกเดินผ่าน | $(\text{Covered Branches} \div \text{Total Branches}) \times 100$                                                                |
| Generation Time             | เวลาที่ใช้ในการสร้าง Test               | วัดระยะเวลาที่ใช้สร้าง Test ของแต่ละวิธี                                                                                         |

ตารางที่ 1 ตัวชี้วัดที่ใช้ในการประเมิน

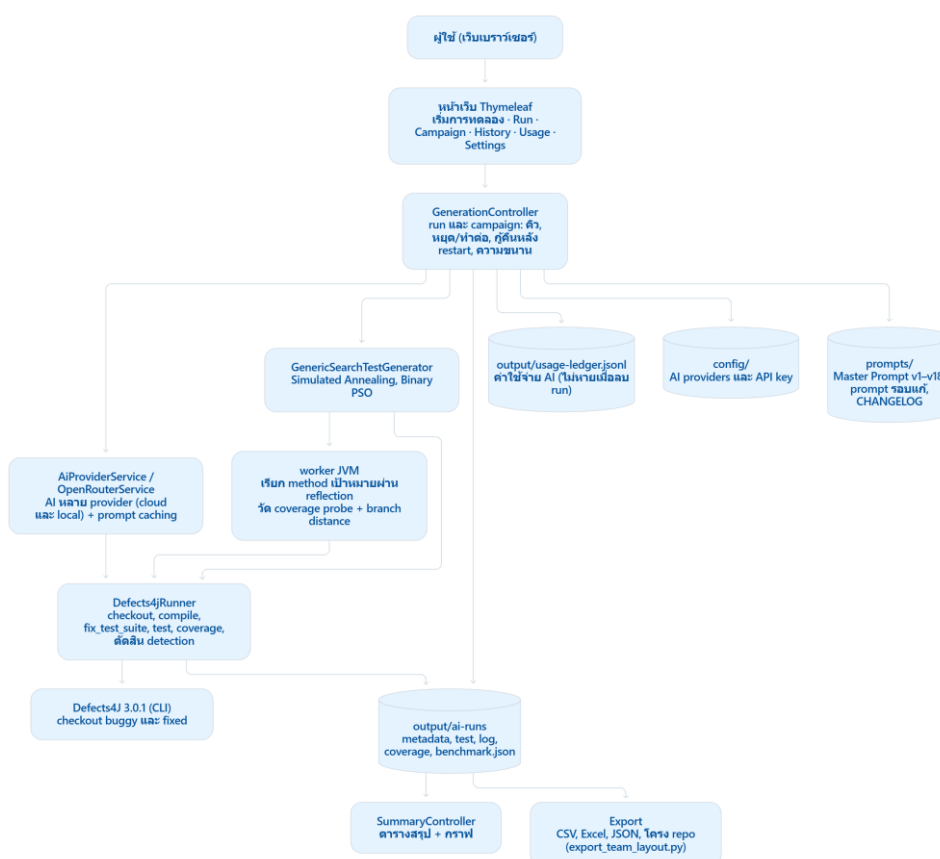
## บทที่ 3

### วิธีการดำเนินงานวิจัย

#### 3.1 ภาพรวมของสถาปัตยกรรมระบบและกระบวนการทำงาน

##### 1. ภาพรวมระบบ

TestBench เป็นเว็บแอป Java/Spring Boot รันใน Docker container ที่ติดตั้ง Defects4J 3.0.1 ไว้แล้ว ผู้ใช้งานผ่านหน้าเว็บ ระบบสร้าง test ด้วย AI และ algorithm แล้วประเมินทุกวิธีด้วยขั้นตอนเดียวกันกับ Defects4J



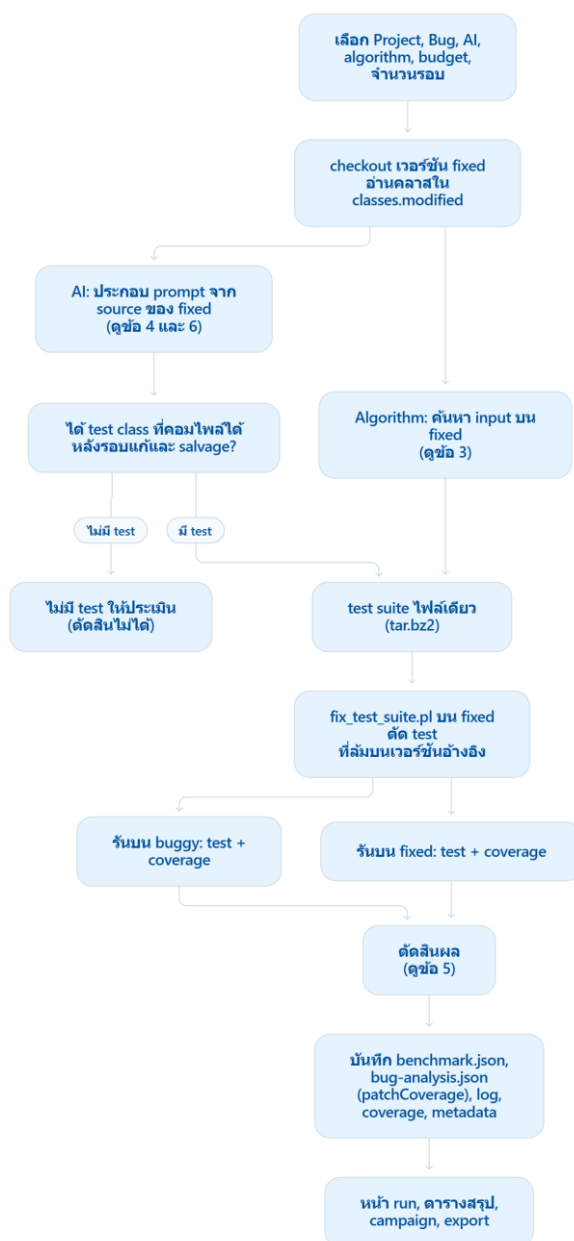
รูปภาพที่ 1 ภาพรวมระบบ

| ส่วน                                    | ไฟล์                                                                                                                                                     |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| หน้าเว็บ,<br>run/campaign,<br>ท่อฝัง AI | src/main/java/edu/kku/sqa/GenerationController.java,<br>src/main/resources/templates/*.html                                                              |
| SA / BPSO<br>และการประกอบ<br>input      | GenericSearchTestGenerator.java, src/main/resources/generator/<br>(GenericFitnessWorker, SearchInputFactory_scaffolding, BranchAgent,<br>BranchRecorder) |
| ประเมินกับ<br>Defects4J                 | Defects4JRunner.java                                                                                                                                     |
| เรียก AI                                | AiProviderService.java, OpenRouterService.java                                                                                                           |
| ตารางสรุป /<br>ส่งออก                   | SummaryController.java, scripts/export_team_layout.py                                                                                                    |
| บัญชีค่าใช้จ่าย AI                      | UsageLedger.java, UsageController.java                                                                                                                   |

ตารางที่ 2 ตารางภาพรวมระบบ

## 2. ขั้นตอนของหนึ่ง run (ต่อหนึ่ง bug ต่อหนึ่งรอบ)

ทั้ง AI และ algorithm ใช้ เวอร์ชัน fixed เป็นเวอร์ชันอ้างอิง (oracle) ตามแนวทางของ Defects4J  
แยกกันสร้าง suite แล้วเข้าขั้นประเมินเดียวกัน



รูปภาพที่ 2 ขั้นตอนของหนึ่ง run (ต่อหนึ่ง bug ต่อหนึ่งรอบ)

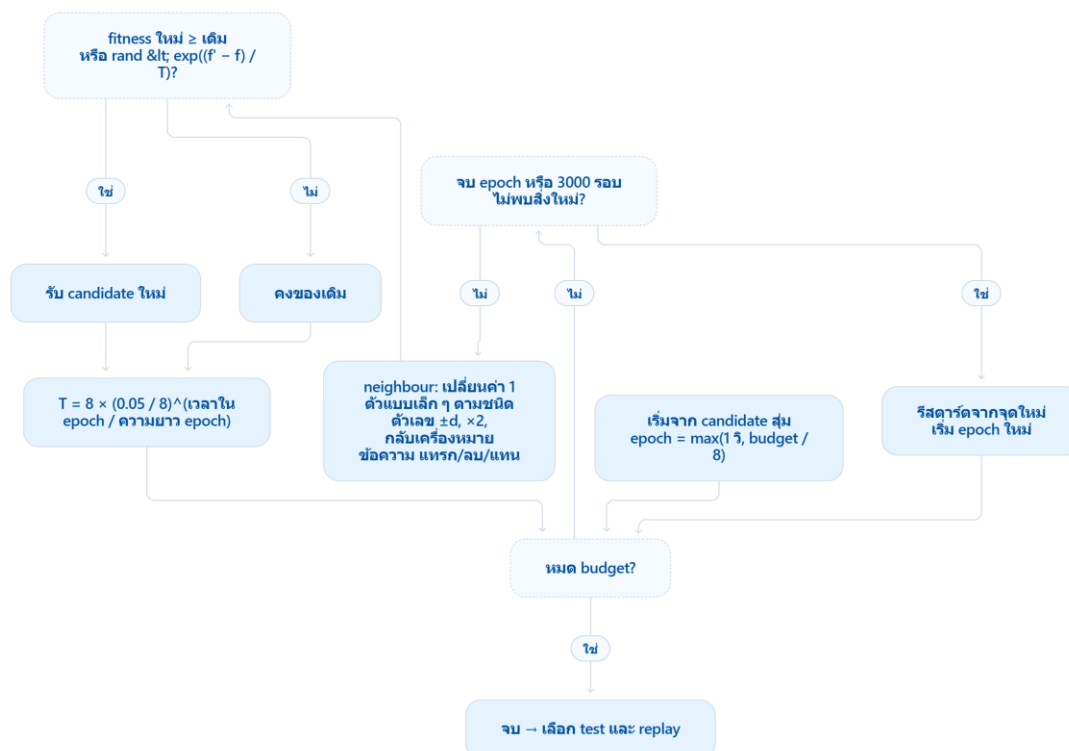
3. Pipeline ของ algorithm (SA และ BPSO ใช้ร่วมกัน) ทั้งสอง algorithm ใช้ "วัตถุดิบ input", fitness และขั้นตอนเก็บ test ชุดเดียวกัน ต่างกันเฉพาะวิธีค้นหา (ข้อ 3.1, 3.2).



รูปภาพที่ 3 Pipeline ของ algorithm (SA และ BPSO ใช้ร่วมกัน)

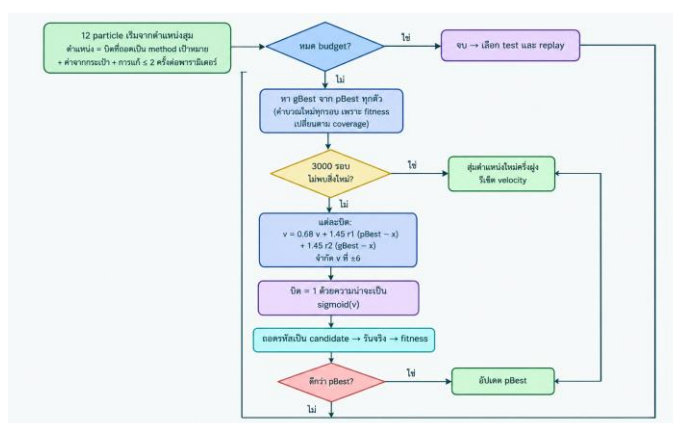
Fitness ของ input หนึ่งตัว = คะแนน coverage probe (JaCoCo) ของคลาสที่แก้ที่ input นั้นวิ่งถึง ถ่วงน้ำหนักให้ probe ที่ยังไม่มี test ไหนไปถึงมีค่ามากที่สุด + ผลรวม  $(1 - \text{branch distance})$  ของเงื่อนไขที่ยังไม่เคยถูกเลี้ยวไปอีกทาง (สูงสุด 30 จุด วัดโดย BranchAgent) + ผลลัพธ์ชนิดใหม่ (ค่า/exception ที่ไม่เคยเห็น)

# - Simulated Annealing



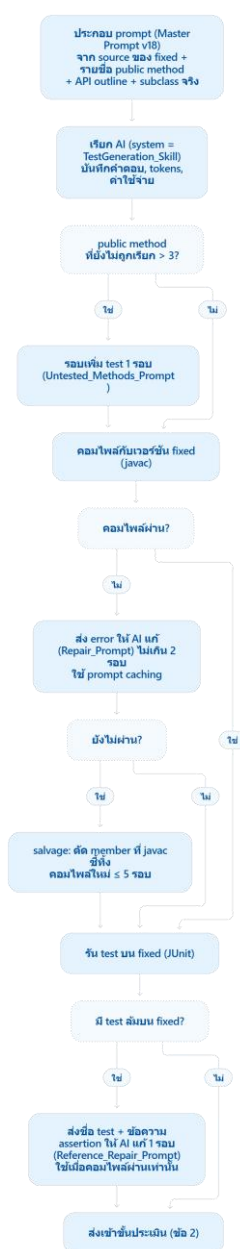
รูปภาพที่ 4 Simulated Annealing

# - Binary PSO



รูปภาพที่ 5 Binary PSO

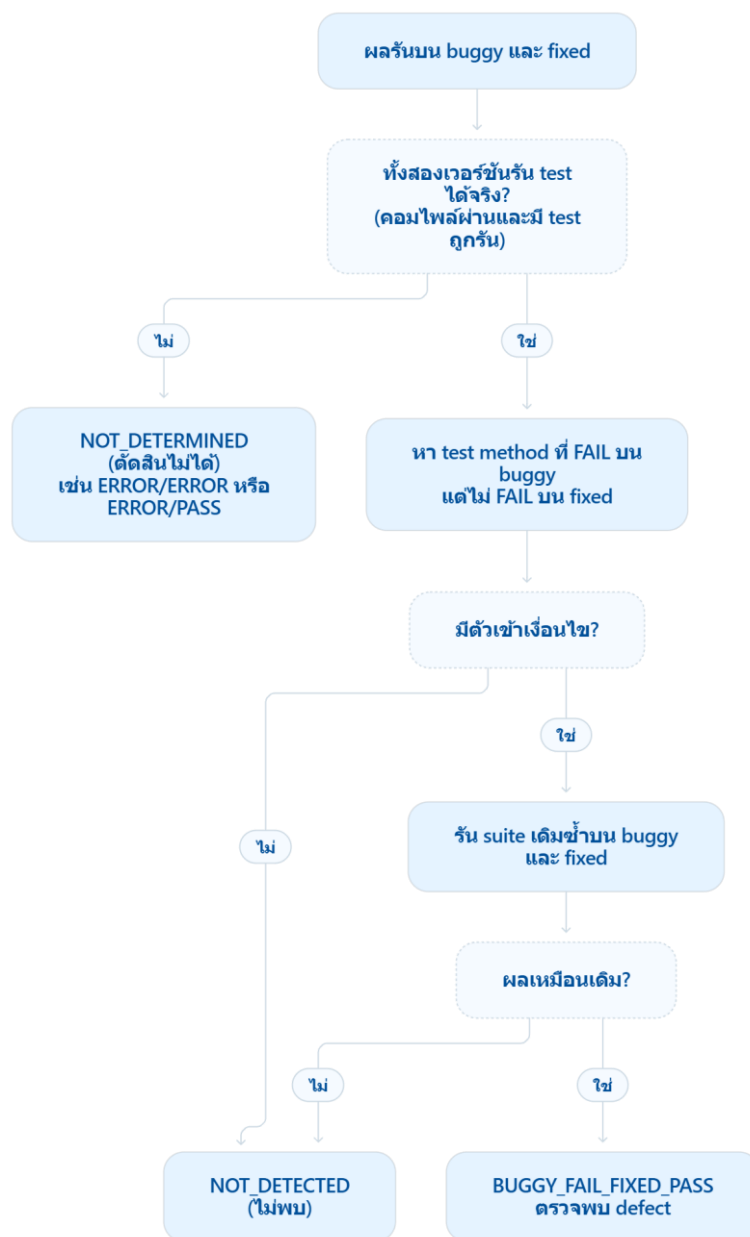
#### 4. Pipeline ของ AI (ต่อ Ai หนึ่งตัว)



รูปภาพที่ 6 Pipeline ของ AI (ต่อ Ai หนึ่งตัว)

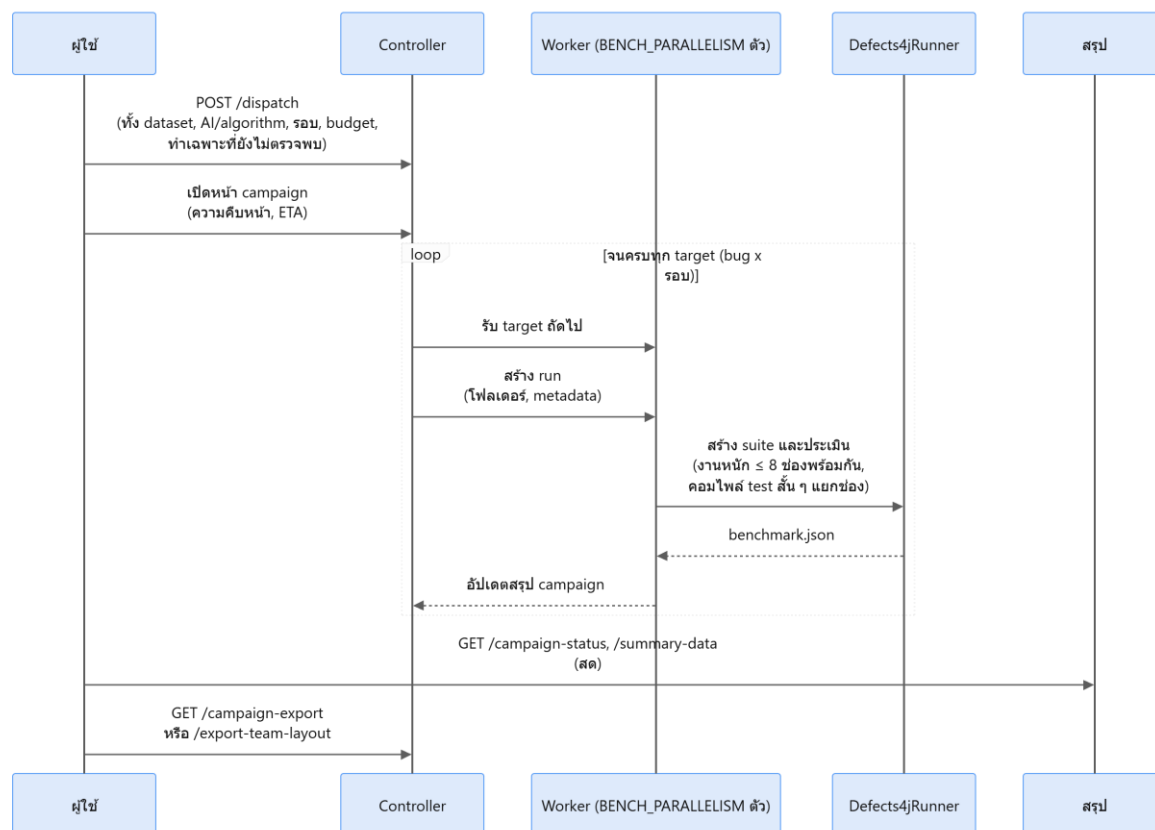
ข้อจำกัดที่ใส่ไว้: รอบแก้จำกัดจำนวน token ตามขนาดคลาส (กันคำตอบยาว)  
และคำตอบที่ถูกตัดที่ขีดจำกัด output จะไม่ถูกนำไปคอมไพล์

## 5. การตัดสินใจผลหลังรัน buggy และ fixed



รูปภาพที่ 7 การตัดสินใจผลหลังรัน buggy และ fixed

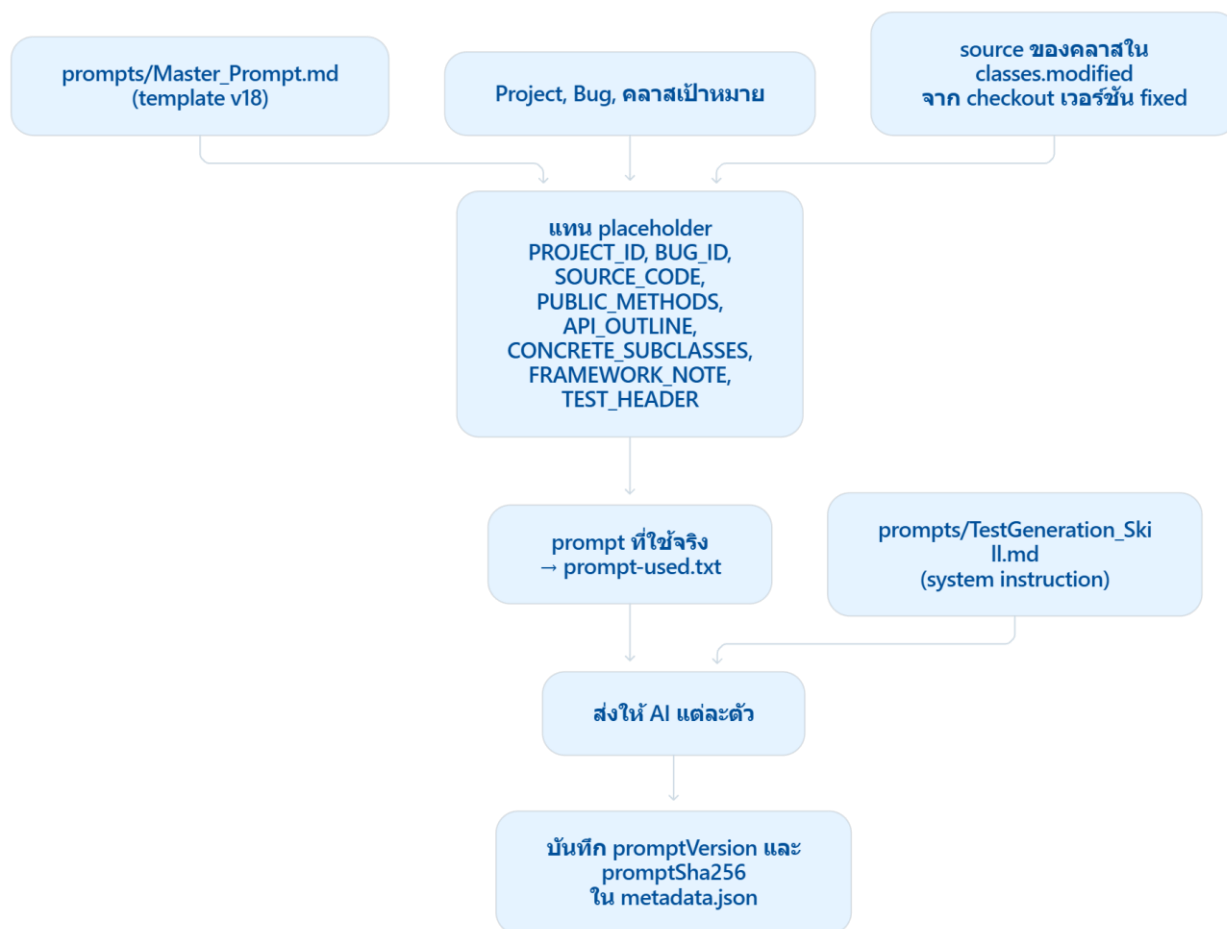
## 6. Campaign (ทั้ง dataset)



รูปภาพที่ 8 Campaign (ทั้ง dataset)

campaign บันทึกสถานะลงดิสก์ทุก target หาก server restart จะกู้คืนและทำต่อจากจุดเดิม  
รันได้สูงสุด 3 campaign พร้อมกัน

## 7. การประกอบ prompt ของ AI



รูปภาพที่ 9 การประกอบ prompt ของ AI

ข้อมูลที่ส่งให้ AI คือ source ของเวอร์ชัน fixed (เวอร์ชันอ้างอิงเดียวกับที่ algorithm ใช้) ไม่มี patch / diff ระหว่างสองเวอร์ชัน และไม่มี test ที่ trigger bug ของ Defects4J

ระบบทดลองที่ใช้ในโครงการพัฒนาด้วยภาษา Java เพื่อรองรับการสร้างและประเมินชุดทดสอบจากวิธีที่นำมาเปรียบเทียบ ได้แก่ Simulated Annealing (SA), Binary Particle Swarm Optimization (BPSO), ChatGPT และ Gemini โดยระบบแบ่งออกเป็นส่วนหลักดังนี้

**GenericSearchTestGenerator** เป็นส่วนที่ทำหน้าที่สร้างชุดทดสอบด้วยอัลกอริทึมการค้นหา SA และ BPSO โดยค้นหาชุดทดสอบที่เหมาะสมตามเกณฑ์ที่กำหนด

**Defects4jRunner** ทำหน้าที่จัดการการทดลองร่วมกับ Defects4J ตั้งแต่การ Checkout โค้ด การรัน Test การวัด Coverage ตลอดจนการเปรียบเทียบผลการทดสอบระหว่างโค้ดเวอร์ชัน Buggy และ Fixed เพื่อพิจารณาว่าชุดทดสอบสามารถตรวจพบบั๊กได้หรือไม่

**GenerationController** ทำหน้าที่ควบคุมการทดลองและจัดการกระบวนการเรียกใช้ AI ได้แก่ ChatGPT และ Gemini เพื่อสร้างชุดทดสอบสำหรับคลาสเป้าหมายของแต่ละบั๊ก

**prompts** เป็นส่วนที่จัดเก็บ Master Prompt สำหรับกำหนดคำสั่งที่ใช้กับ ChatGPT และ Gemini โดยมี Prompt หลายเวอร์ชัน รวมถึง Repair Prompt ซึ่งใช้สำหรับแก้ไข Test เมื่อพบปัญหาจากการ Compile หรือการรัน Test

**scripts** เป็นส่วนที่ใช้จัดการและส่งออกผลการทดลองให้อยู่ในโครงสร้างโฟลเดอร์ที่กำหนด เพื่อให้สามารถรวบรวมผลและนำไปวิเคราะห์เปรียบเทียบได้อย่างเป็นระบบ

สำหรับกระบวนการทดลอง **1 บั๊กต่อ 1 วิธี** เริ่มจากการ Checkout Source Code ของบั๊กจาก Defects4J ทั้งเวอร์ชัน Buggy และ Fixed พร้อมระบุคลาสที่มีการแก้ไข จากนั้นระบบจะสร้าง Test ด้วยวิธีที่กำลังทดลอง ได้แก่ SA, BPSO, ChatGPT หรือ Gemini และบันทึก Test ที่สร้างได้ในรูปแบบ JUnit Test Suite

หลังจากสร้าง Test แล้ว ระบบจะ Compile และรัน Test บนโค้ดทั้งเวอร์ชัน Fixed และ Buggy จากนั้นนำผลการรันมาเปรียบเทียบเพื่อตัดสินว่าสามารถตรวจพบบั๊กได้หรือไม่ตามหลักเกณฑ์ที่กำหนดไว้ในหัวข้อ 3.5 นอกจากนี้ ระบบจะวัดค่า Line Coverage และ Condition/Branch Coverage ของคลาสที่ถูกแก้ไข พร้อมบันทึกเวลาที่ใช้ในการสร้าง Test และผลการทดลองที่เกี่ยวข้อง เพื่อนำข้อมูลจากแต่ละวิธีมารวบรวม วิเคราะห์ และเปรียบเทียบประสิทธิภาพในขั้นตอนต่อไป

### 3.2 การจัดเตรียมชุดข้อมูลและการคัดเลือก Java Projects จาก Defects4J

การทดลองใช้ Defects4J ซึ่งมี 17 Project (Chart, Cli, Closure, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang, Math, Mockito, Time) จำนวนบั๊กที่ใช้ทดสอบในผลการทดลองรวมคือ 854 บั๊กต่อวิธี โดยทุกวิธีทดสอบกับชุดบั๊กเดียวกัน เพื่อให้เปรียบเทียบกันได้โดยตรง สำหรับแต่ละบั๊ก Test

จะมุ่งไปที่คลาสที่ถูกแก้ตามข้อมูลของ Defects4J (classes.modified) และ Coverage  
วัดเฉพาะคลาสเหล่านี้

### 3.3 การออกแบบและพัฒนา Automated Test Case Generation (SA และ BPSO)

#### Candidate Solution และ Fitness

หนึ่ง Candidate Solution แทนชุด Test หนึ่งชุด ซึ่ง Fitness ประเมินจากการรัน Test กับ Target Class ตามเกณฑ์ 4 ข้อ ได้แก่ Line (บรรทัดที่ถูกรัน) Branch (เงื่อนไขและเส้นทางที่ถูกเดินผ่าน) Exception (พฤติกรรมที่เกี่ยวข้องกับ Exception) และ Weak Mutation (ช่วยผลักดันการค้นหาไปทาง Test ที่แยกพฤติกรรมของโปรแกรมได้) ส่วนของโค้ดที่ยังไม่มี Test ใดเคยไปถึงจะได้คะแนนสูง และเงื่อนไขที่เกือบจะเข้าได้ (Branch Distance) ก็ได้คะแนนบางส่วน ทำให้การค้นหามีทิศทางให้เดินตาม Fitness สูงขึ้นหมายถึงชุด Test มีคุณภาพตามเป้าหมายมากขึ้น

#### การออกแบบ SA

เริ่มจาก Candidate Solution แบบสุ่ม แล้วสร้างคำตอบข้างเคียงด้วยการปรับเล็กน้อย (เปลี่ยนค่า 1 ถึง 2 ตำแหน่ง)

ถ้า Fitness ดีขึ้นจะรับทันที ถ้าแย่ลงจะรับด้วยความน่าจะเป็น  $P = e^{-(\Delta/T)}$

Temperature เริ่มที่ 8 และลดลงตามเวลาที่ใช้ไปจนถึง 0.05

เมื่อไม่พบผลใหม่เป็นเวลานาน จะเริ่มใหม่จากจุดสุ่ม

#### การออกแบบ BPSO

แต่ละ Particle มีตำแหน่งเป็นบิต โดย 1 คือเลือก Test และ 0 คือไม่เลือก ตัวอย่างเช่นเมื่อมี Candidate Test 4 ตัว ตำแหน่ง 1 0 1 1 หมายถึงชุดที่เลือก T1, T3 และ T4 แต่ละ Particle เก็บ pBest และอ้างอิง gBest ของทั้งกลุ่ม

ความเร็วคำนวณจาก  $v = w \cdot v + c1 \cdot r1 \cdot (pBest - x) + c2 \cdot r2 \cdot (gBest - x)$  ในระบบกำหนด  $w = 0.68$ ,  $c1 = c2 = 1.45$  และจำกัดความเร็วที่  $\pm 6$

แปลงความเร็วเป็นความน่าจะเป็นด้วย Sigmoid แล้วสุ่มค่าบิตตามนั้น

ถ้าไม่พบผลใหม่เป็นเวลานาน จะสุ่มตำแหน่งใหม่ให้ Particle บางส่วนเพื่อกระจายการค้นหา

ค่าพารามิเตอร์ของ SA และ BPSO ข้างต้นเป็นค่าที่กำหนดไว้ในโค้ดของระบบ  
ไม่ได้ผ่านการปรับปรุงเปรียบเทียบเพิ่มเติม

3.4 การออกแบบ Prompt Engineering สำหรับ AI-Assisted Testing (ChatGPT และ Gemini)

ฝั่ง AI ทำเป็น Pipeline เริ่มจากดึง Context ของแต่ละบั๊ก ได้แก่ Target Class, Source Code, ข้อมูลบั๊ก (Bug Information) และข้อกำหนดการทดสอบ (Testing Requirements) แล้วนำไปใส่ใน Master Prompt ซึ่งมีโครงสร้าง 5 ส่วน

ส่วนของ Prompt

| หน้าที่        | รายละเอียด                                                            |
|----------------|-----------------------------------------------------------------------|
| Role           | กำหนดบทบาทให้ AI เป็นผู้ผู้เชี่ยวชาญการเขียน Unit Test                |
| Task           | ระบุงานที่ต้องทำ คือสร้าง JUnit Test สำหรับคลาสเป้าหมาย               |
| Source Context | ใส่ Source Code และข้อมูลบั๊กของแต่ละกรณี                             |
| Constraints    | กำหนดข้อจำกัด เช่น Test ต้อง Compile ได้ และใช้ได้กับ API ของโค้ดจริง |
| Output Format  | กำหนดรูปแบบผลลัพธ์ให้เป็นไฟล์ JUnit ที่สามารถนำไปรันได้               |

ตารางที่ 3 การออกแบบ Prompt Engineering สำหรับ AI-Assisted Testing

ใช้ Prompt แนวเดียวกันในระดับ Project แล้วแทนข้อมูลของแต่ละบั๊กลงใน Context เมื่อได้ Test จาก AI จะ Compile บนเวอร์ชัน Fixed ก่อน จากนั้นรันทั้ง Fixed และ Buggy และวัด Coverage ChatGPT และ Gemini ใช้ผ่าน KKU API ด้วยการตั้งค่า Prompt แบบเดียวกัน

3.5 สภาพแวดล้อมและระเบียบวิธีการทดลอง (Experimental Setup)

3.5.1 หลักการตัดสินผลเปรียบเทียบ Buggy กับ Fixed

ระบบรัน Test บนทั้งสองเวอร์ชัน แล้วตัดสินตามตารางที่ 3.1 โดยตัดสินเป็นรายตัว Test Method และรันซ้ำเพื่อยืนยัน ถ้าผลไม่นิ่งจะไม่นับว่าตรวจพบ นอกจากนี้ใช้สคริปต์ fix\_test\_suite ของ Defects4J ตัด Test ที่ไม่ผ่านบน Fixed ออกก่อนวัดผล เพราะมักเกิดจากค่าคาดหวังที่เขียนผิด ไม่ใช่การเจอบั๊กจริง

| Buggy               | Fixed     | ผลตัดสิน                    | ความหมาย                                                                                |
|---------------------|-----------|-----------------------------|-----------------------------------------------------------------------------------------|
| PASS                | PASS      | NOT_DETECTED                | Test<br>ไม่ได้แตะจุดที่พฤติกรรมต่างกัน                                                  |
| FAIL                | PASS      | BUGGY_FAIL_FIXED_PASS       | <b>ตรวจพบ</b> ถ้ายืนยันซ้ำได้                                                           |
| PASS                | FAIL      | NOT_DETECTED                | พฤติกรรมต่างกันจริง<br>แต่ไม่นับว่าเจอบั๊ก                                              |
| FAIL                | FAIL      | NOT_DETECTED<br>เป็นส่วนมาก | มักเป็น Test ผิดเอง แต่ถ้ามี<br>Test ที่ล้มเฉพาะบน Buggy<br>และยืนยันได้ ก็นับว่าตรวจพบ |
| ERROR /<br>NO_TESTS | อะไรก็ได้ | NOT_DETERMINED              | ไม่มีผลที่ใช้เทียบกันได้                                                                |

ตารางที่ 4 ผลตัดสินจากการเทียบผล Buggy กับ Fixed

| รายการ                   | ค่าที่ใช้                                      |
|--------------------------|------------------------------------------------|
| Dataset                  | Defects4J (Java Projects ที่มีบั๊กจริง)        |
| วิธีที่เปรียบเทียบ       | SA, BPSO, ChatGPT, Gemini                      |
| Search Budget (SA, BPSO) | 30 วินาที                                      |
| Population               | 30                                             |
| Random Seed              | 20260930                                       |
| เกณฑ์การค้นหา            | LINE, BRANCH, EXCEPTION, WEAKMUTATION          |
| จำนวนรอบ                 | 1 Run ต่อบั๊กต่อวิธี                           |
| Environment              | Java, Defects4J, JUnit, เครื่องมือวัด Coverage |

ตารางที่ 5 ค่ากำหนดการทดลอง

หลักการควบคุมการทดลองคือใช้ Dataset เดียวกัน ตั้งค่าเดียวกัน และประเมินด้วยเกณฑ์เดียวกันกับทุกวิธี

## บทที่ 4

### ผลการศึกษาและวิเคราะห์ผล

#### 4.1 ผลการทดสอบระดับความครอบคลุมของโค้ด (Code Coverage Evaluation)

ด้าน Line Coverage Gemini ได้ค่าเฉลี่ยสูงสุดที่ 70.10% ตามด้วย ChatGPT 68.70% BPSO 64.20% และ SA 61.80% ด้าน Condition Coverage ลำดับเป็นแบบเดียวกัน คือ Gemini 60.20% ChatGPT 58.40% BPSO 52.10% และ SA 49.60% จะเห็นว่าวิธีที่ใช้ AI ได้ Coverage สูงกว่าอัลกอริทึมค้นหาทั้งสองตัวในทั้งสองตัวชี้วัด และ Condition Coverage ต่ำกว่า Line Coverage ในทุกวิธี แสดงว่าการเข้าถึงเงื่อนไขและเส้นทางต่าง ๆ ทำได้ยากกว่าการรันให้ครบบรรทัด

| วิธี    | บั๊กที่ทดสอบ | บั๊กที่ตรวจพบ | Detection Rate | Avg. Line Coverage | Avg. Condition Coverage |
|---------|--------------|---------------|----------------|--------------------|-------------------------|
| SA      | 854          | 281           | 32.90%         | 61.80%             | 49.60%                  |
| BPSO    | 854          | 296           | 34.70%         | 64.20%             | 52.10%                  |
| ChatGPT | 854          | 327           | 38.30%         | 68.70%             | 58.40%                  |
| Gemini  | 854          | 315           | 36.90%         | 70.10%             | 60.20%                  |

ตารางที่ 6 ผลการทดลองของทั้ง 4 วิธี (ทดสอบบน 854 บั๊กเท่ากัน)

Detection Rate คำนวณจากจำนวนบั๊กที่ตรวจพบหารจำนวนบั๊กที่ทดสอบ ตัวอย่างเช่น SA =  $281 \div 854 = 32.90\%$

#### 4.2 ผลการทดสอบความสามารถในการตรวจจับข้อผิดพลาด (Fault Detection Rate)

ChatGPT ตรวจพบบั๊กมากที่สุด 327 บั๊ก (38.30%) ตามด้วย Gemini 315 บั๊ก (36.90%) BPSO 296 บั๊ก (34.70%) และ SA 281 บั๊ก (32.90%) เมื่อเทียบกับ SA เป็นฐาน ผลต่างของแต่ละวิธีเป็นดังตารางที่

4.2

|         |     |      |      |       |
|---------|-----|------|------|-------|
| BPSO    | +15 | +1.8 | +2.4 | +2.5  |
| ChatGPT | +46 | +5.4 | +6.9 | +8.8  |
| Gemini  | +34 | +4.0 | +8.3 | +10.6 |

ตารางที่ 7 ผลต่างของแต่ละวิธีเมื่อเทียบกับ SA

### 4.3 การเปรียบเทียบระยะเวลาและทรัพยากรในการสร้าง Test Case

ข้อมูลส่วนนี้ยังไม่เพียงพอ ข้อมูลที่มีระบุเพียงว่า SA และ BPSO ถูกจำกัดเวลาค้นหาที่ 30 วินาทีต่อบั๊ก ส่วนเวลาสร้าง Test กรณีของแต่ละวิธี (รวมเวลา Generate, Compile และรัน) และทรัพยากรที่ใช้ ยังไม่มีตัวเลขสรุป จึงยังเปรียบเทียบไม่ได้ ต้องการข้อมูล Generation Time เฉลี่ยต่อบั๊กของทั้ง 4 วิธีเพื่อเติมในหัวข้อนี้

### 4.4 วิเคราะห์เปรียบเทียบเชิงลึกระหว่าง AI-Assisted Testing กับ Optimization Algorithms

1) AI ให้ผลสูงกว่าอัลกอริทึมในการทดลองนี้ ChatGPT และ Gemini ตรวจพบบั๊กและได้ Coverage สูงกว่า SA และ BPSO ทั้งหมด ซึ่งสอดคล้องกับข้อสังเกตว่า AI อ่านบริบทของ Source Code และข้อมูลบั๊กได้โดยตรง ขณะที่ SA และ BPSO ต้องค้นหาภายในเวลา 30 วินาที จึงอาจสำรวจ Search Space ได้ไม่ครบ

2) Coverage สูงไม่ได้แปลว่าตรวจจับบั๊กได้มากกว่า Gemini มี Coverage สูงที่สุดทั้ง Line และ Condition แต่ตรวจพบบั๊กน้อยกว่า ChatGPT อยู่ 12 บั๊ก ลำดับด้าน Coverage (Gemini > ChatGPT) จึงสลับกับลำดับด้านการตรวจพบ (ChatGPT > Gemini) เหตุผลคือ Test ต้องไปโดนจุดที่บั๊กอยู่ และตรวจค่าให้ตรงจุดด้วย ไม่ใช่แค่รันผ่านโค้ดให้มาก

3) BPSO สูงกว่า SA เล็กน้อย BPSO ได้ค่าสูงกว่า SA ในทุกตัวชี้วัด แต่ส่วนต่างไม่มาก (ตรวจพบเพิ่ม 15 บั๊ก หรือ 1.8 จุดเปอร์เซ็นต์) และการทดลองรัน 1 รอบต่อบั๊ก  
 จึงยังสรุปไม่ได้ว่าความต่างนี้เกิดจากวิธีเองหรือจากความสุ่ม

4) ข้อควรระวังในการตีความ การทดลองรัน 1 รอบต่อบั๊กต่อวิธีและยังไม่ได้ทดสอบทางสถิติ ผลแต่ละวิธีอาจขึ้นกับ Project และชนิดของบั๊ก และทั้ง AI กับอัลกอริทึมใช้เวอร์ชัน Fixed เป็นแหล่งอ้างอิงค่าคาดหวัง ซึ่งเป็นแนวทางที่ใช้กันกับ Defects4J แต่ต้องระบุไว้ให้ชัดเมื่อตีความผล

## บทที่ 5

### สรุปผลการศึกษาและข้อเสนอแนะ

#### 5.1 สรุปผลการดำเนินงาน

โครงการนี้เปรียบเทียบการสร้าง Test Case อัตโนมัติ 4 วิธี ได้แก่ SA, BPSO, ChatGPT และ Gemini บนบักจริงใน Defects4J จำนวน 854 บัก ด้วย Dataset และการตั้งค่าเดียวกัน ผลสรุปได้ดังนี้ ทั้ง AI และอัลกอริทึมค้นหาสร้าง Test ที่นำไปตรวจจับบักได้จริง โดย ChatGPT ตรวจพบสูงสุด 327 บัก (38.30%) ตามด้วย Gemini 315 บัก (36.90%) BPSO 296 บัก (34.70%) และ SA 281 บัก (32.90%) Gemini ได้ Coverage สูงสุด (Line 70.10% และ Condition 60.20%) ตามด้วย ChatGPT, BPSO และ SA ตามลำดับ Coverage สูงไม่ได้หมายความว่าตรวจพบบักมากที่สุด เพราะวิธีที่ได้ Coverage สูงสุดไม่ใช่วิธีที่ตรวจพบบักมากที่สุด จึงควรพิจารณาหลายตัวชี้วัดร่วมกัน BPSO ให้ผลสูงกว่า SA เล็กน้อยในทุกตัวชี้วัด

ไม่มีวิธีใดวิธีเดียวที่เหมาะสมกับบักทุกประเภท ประสิทธิภาพขึ้นกับ Project บัก และวิธีสร้าง Test การเลือกวิธีจึงควรพิจารณา Bug Detection, Coverage และ Generation Time ไปพร้อมกัน

#### 5.2 ข้อดีและข้อจำกัดของแต่ละแนวทาง

| วิธี    | ข้อดี                                                                                   | ข้อจำกัด                                                                                          |
|---------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| SA      | หลักการตรงไปตรงมา<br>ยอมรับคำตอบที่แย่งชั่วคราวเพื่อหลีกเลี่ยง<br><b>Local Optimum</b>  | ได้ผลต่ำสุดในการทดลองนี้<br>และมีเวลาค้นหา 30 วินาที<br>จึงอาจสำรวจ <b>Search Space</b><br>ไม่ครบ |
| BPSO    | ใช้ <b>Particle</b><br>หลายตัวเรียนรู้จากตัวเองและจากกลุ่ม<br>ได้ผลสูงกว่า SA เล็กน้อย  | ยังต่ำกว่า AI ทั้งสองตัว<br>และถูกจำกัดด้วยเวลาค้นหา 30<br>วินาทีเช่นกัน                          |
| ChatGPT | ตรวจพบบักสูงสุดในการทดลอง สร้าง Test<br>จาก <b>Source Code</b><br>และข้อมูลบักได้โดยตรง | Test อาจ Compile ไม่ผ่านบน<br>Fixed และ Source Code<br>ขนาดใหญ่ทำให้ Prompt ใหญ่                  |
| Gemini  | <b>Coverage ทั้ง Line และ Condition<br/>สูงที่สุด</b>                                   | ตรวจพบบักน้อยกว่า ChatGPT<br>แม้ Coverage สูงกว่า                                                 |

|  |  |                                                      |
|--|--|------------------------------------------------------|
|  |  | และมีข้อจำกัดเรื่อง Compile และ Context เช่นเดียวกัน |
|--|--|------------------------------------------------------|

ตารางที่ 7 ข้อดีและข้อจำกัดของแต่ละแนวทาง

### 5.3 ปัญหาและอุปสรรคในการดำเนินงาน

AI สร้าง Test ที่ Compile ไม่ผ่าน เช่น เรียก Method ไม่ตรงกับ Source ใช้ Constructor ไม่ถูกต้อง Import หรือ Class ผิด ทำให้ Test ไม่สามารถ Compile บน Fixed Version ได้ Source Context ขนาดใหญ่ บาง Project มี Source Code ใหญ่ ทำให้ Prompt ใหญ่และการสร้าง Test ซับซ้อนขึ้น Search Budget จำกัด SA และ BPSO มีเวลาดำเนินการ 30 วินาที ทำให้ Search Space อาจดูสำรวจไม่ครบ Environment การทดลองต้องพึ่ง Java, Defects4J, Dependencies, JUnit และ Coverage Tools ซึ่ง Environment ที่ต่างกันอาจส่งผลต่อการรัน Test ต้นทุนการคำนวณ จำนวน Projects  $\times$  Bugs  $\times$  4 วิธี ทำให้ต้อง Generate, Compile และรัน Test จำนวนมาก

### 5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อยอดในอนาคต

ทำซ้ำหลายรอบ รันหลายรอบด้วย Random Seed ต่างกัน แล้วทดสอบทางสถิติ เพื่อยืนยันว่าความต่างระหว่างวิธี โดยเฉพาะ BPSO กับ SA ไม่ได้เกิดจากความสุ่ม

เพิ่ม Search Budget ลองเพิ่มเวลาดำเนินการของ SA และ BPSO และดูว่าผลดีขึ้นเพียงใด ลดปัญหา Compile ผัง AI เพิ่มขึ้นตอนส่งข้อผิดพลาดจากการ Compile กลับให้ AI แก้ Test แล้วรันซ้ำผสมแนวทาง ใช้ Test ที่ AI สร้างเป็นจุดเริ่มต้นให้ SA หรือ BPSO ปรับปรุงต่อ เพื่อรวมจุดเด่นของทั้งสองฝั่ง (เป็นข้อเสนอแนะ ยังไม่ได้ทดลอง)วิเคราะห์รายโปรเจกต์ แยกผลตาม Project เพื่อดูว่าแต่ละวิธีเหมาะกับ Project แบบใด บันทึกเวลาสร้าง Test อย่างเป็นระบบ เพื่อนำมาเปรียบเทียบด้านเวลาและทรัพยากรได้ครบทุกวิธี

## เอกสารอ้างอิง

Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering and 13th European Software Engineering Conference* (pp. 416–419). ACM. <https://doi.org/10.1145/2025113.2025179>

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (pp. 437–440). ACM. <https://doi.org/10.1145/2610384.2628055>

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95—International Conference on Neural Networks* (Vol. 4, pp. 1942–1948). IEEE. <https://doi.org/10.1109/ICNN.1995.488968>

Kennedy, J., & Eberhart, R. C. (1997). A discrete binary version of the particle swarm optimization algorithm. In *Proceedings of the 1997 IEEE International Conference on Systems, Man, and Cybernetics* (Vol. 5, pp. 4104–4108). IEEE. <https://doi.org/10.1109/ICSMC.1997.637339>

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>